Controllers

app/Http/Controllers/CmcController.php
app/Http/Controllers/GdfController.php
app/Http/Controllers/GdrController.php
app/Http/Controllers/LangController.php
app/Http/Controllers/TdbController.php

app/Http/Controllers/CmcController.php

```php
<?php

namespace App\Http\Controllers;

use App\Enum\UserCategorieEnum;
use App\Models\User;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Hash;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class CmcController extends Controller
{
    /**
     * On retourne la page Creation / Modification de comptes
     *
     * @return \Illuminate\Contracts\View\View returned
     */
    public function index()
    {
        // On recuperes tout les clients si on est admin ou gestionnaire
        if (in_array(Auth::user()->categorie->value, ['Gestionnaire',
'Administrateur']))
        {
            $clients = User::where('id','!=',Auth::user()->id)
                            ->orderByRaw("FIELD(categorie , 'Client', 'Gestionnaire',
'Administrateur') ASC")
                            ->get();
        }

        return view('app.creation_modification_comptes')
                ->with('clients', $clients ?? null);
    }

    /**
     * Ajouter un utilisateur
     *
     * @param Request $request
     * @return \Illuminate\Http\RedirectResponse returned
     */
    public function addUser(Request $request)
    {
        $request->validate([
```

```
            'prenom'            => ['required', 'string', 'max:255'],
            'nom'               => ['required', 'string', 'max:255'],
            'email'             => ['required', 'string', 'email', 'max:255',
'unique:users'],
            'password'          => ['required', 'confirmed', Password::defaults()],
            'categorie'         => ['required', Rule::in(UserCategorieEnum::ADMIN-
>value,UserCategorieEnum::GESTION->value,UserCategorieEnum::CLIENT->value)]
        ]);

        $user = new User();
        $user->email                = $request->email;
        $user->nom                  = $request->nom;
        $user->prenom               = $request->prenom;
        $user->password             = Hash::make($request->password);
        $user->categorie            = $request->categorie;
        $user->save();

        return back()->with('statutAddUser',
trans('creation_modification_comptes.StatutaddUser'));
    }

    /**
     * Mise à jour des informations d'un utilisateur
     *
     * @param Request $request
     * @param [type] $id
     * @return \Illuminate\Http\RedirectResponse returned
     */
    public function updateUser(Request $request, $id)
    {
        // S'assure que la demande est faisable
        try {
            $user = User::findOrFail($id);
            $request->validate([
                'prenom'            => ['nullable', 'string', 'max:255'],
                'nom'               => ['nullable','string', 'max:255'],
                'email'             => ['nullable','string', 'email', 'max:255',
Rule::unique('users')->ignore($id)],
                'password'          => ['nullable','confirmed',Password::defaults()],
                'categorie'         => ['nullable',Rule::in(UserCategorieEnum::ADMIN-
>value,UserCategorieEnum::GESTION->value,UserCategorieEnum::CLIENT->value)]
            ]);

        } catch (\Throwable $th)
        {
            return back()->with('statutErreur',
trans('creation_modification_comptes.StatutError'));
        }

        $user->email                = $request->email ?? $user->email;
        $user->nom                  = $request->nom ?? $user->nom;
        $user->prenom               = $request->prenom ?? $user->prenom;
        $user->password             = Hash::make($request->password) ?? $user-
>password;
        $user->categorie            = $request->categorie ?? $user->categorie;
        // On recupere le nom de l'utilisateur
        $userName = $user->prenom . " " . $user->nom;

        $user->save();
```

```
        return back()->with('statutDeleteUpdate',
trans('creation_modification_comptes.StatutupdateUser', ['nom' => $userName]));
    }

    /**
     * On renvoi les données necessaire pour l'affichage
     * des informations d'un utilisateur séléctionner
     *
     * @param [type] $id
     * @return \Illuminate\Http\JsonResponse returned
     */
    public function showUpdateUser($id)
    {
        $data = User::findOrFail($id);
        return response()->json($data);
    }

    /**
     * Mettre à jour le mot de passe du compte
     *
     * @param Request $request
     * @return
     */
    public function update(Request $request)
    {
        $request->validate([
            'password' => ['required', Password::defaults(), 'confirmed'],
        ]);

        $request->user()->update([
            'password' => Hash::make($request->password)
        ]);

        return back()->with('statut',
trans('creation_modification_comptes.StatutupdateMdp'));
    }

    /**
     *
     * Pouvoir supprimer un utilisateur
     *
     * @category Administrateur
     * @param [type] $id
     * @return \Illuminate\Http\RedirectResponse returned
     */
    public function delete($id)
    {
        // S'assure que la demande est faisable
        try {
            $user = User::findOrFail($id);
        } catch (\Throwable $th)
        {
            return back()->with('statutErreur',
trans('creation_modification_comptes.StatutError'));
        }
```

```
        // On recupere le nom de l'utilisateur
        $userName = $user->prenom . " " . $user->nom;

        // On supprime l'utilisateur
        $user->delete();

        return back()->with('statutDeleteUpdate',
trans('creation_modification_comptes.StatutdeleteUser', ['nom' => $userName]));
    }

    /**
     * Mettre à jour les informations personnelles de l'administrateur
     *
     * @param Request $request
     * @return \Illuminate\Http\RedirectResponse returned
     */
    public function updateInfoPerso(Request $request)
    {
        $request->validate([
            'prenom'            => ['nullable', 'string', 'max:255'],
            'nom'               => ['nullable','string', 'max:255'],
            'email'             => ['nullable','string', 'email', 'max:255',
Rule::unique('users')->ignore(Auth::user()->id)],
        ]);

        $user = User::findOrFail(Auth::user()->id);
        $user->email = $request->email ?? $user->email;
        $user->nom = $request->nom ?? $user->nom;
        $user->prenom = $request->prenom ?? $user->prenom;
        $user->save();

        return back()->with('statutInfoPerso',
trans('creation_modification_comptes.StatutInfoPerso'));
    }

}
```

app/Http/Controllers/GdfController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Marque;
use App\Models\Modele;
use App\Models\Location;
use App\Models\Vehicule;
use Illuminate\Http\Request;
use App\Enum\VehiculeTypeEnum;
use App\Enum\UserCategorieEnum;
use Illuminate\Validation\Rule;
use App\Enum\VehiculeStatutEnum;
use Illuminate\Support\Facades\Auth;
```

```php
use App\Enum\VehiculeTransmissionEnum;
use Illuminate\Support\Facades\Storage;

class GdfController extends Controller
{
    /**
     * On retourne la page Gestion de la flotte
     *
     * @return \Illuminate\Contracts\View\View
     */
    public function index()
    {
        if (Auth::user()->categorie->value == UserCategorieEnum::GESTION->value) {
            $marques = Marque::all();
            $modeles = Modele::all();
        }

        // Vue d'ensemble des véhicules
        $vehicules = Vehicule::orderByRaw("FIELD(statut , 'Disponible', 'En
révision', 'Hors d'usage', 'Loué') ASC")
                                ->get();

        $locations = Location::all();
        return view('app.gestion_de_la_flotte')
                ->with('marques', $marques ?? null)
                ->with('modeles', $modeles ?? null)
                ->with('vehicules', $vehicules ?? null)
                ->with('locations', $locations ?? null);
    }

    /**
     * Ajouter une voiture
     *
     * @category Gestionnaire
     * @param Request $request
     * @return \Illuminate\Http\RedirectResponse
     */
    public function addCar(Request $request)
    {
        $request->validate([
            'vehicule_image'    =>
'required|image|mimes:jpeg,png,jpg,gif,svg|max:2048',
            'id'                => ['required', 'string', 'max:255',
'unique:vehicules,id'],
            'tarifj'            => ['required', 'numeric'],
            'marque'            => ['required', 'string', 'max:255'],
            'modele'            => ['required', 'string', 'max:255'],
            'type'              => ['required', Rule::in(VehiculeTypeEnum::BERLINE-
>value,VehiculeTypeEnum::BREAK->value,VehiculeTypeEnum::SUV-
>value,VehiculeTypeEnum::MONOSPACE->value)],
            'statut'            => ['required', Rule::in(VehiculeStatutEnum::DISPO-
>value,VehiculeStatutEnum::REVISION->value,VehiculeStatutEnum::HORSU-
>value,VehiculeStatutEnum::LOUER->value)],
            'transmission'      => ['required',
Rule::in(VehiculeTransmissionEnum::MANUEL->value,VehiculeTransmissionEnum::AUTO-
>value)],
        ]);

        // Trouver le modele et la marque sinon les crées pour la création du
vehicule
```

```php
        $marque = Marque::firstOrCreate(['libelle' => $request->marque]);
        $modele = Modele::firstOrCreate(['libelle' => $request->modele]);

        // On enregistre la nouvelle route ainsi que la nouvelle image
        $img_vehicule_path = $request->vehicule_image->store('img/vehicules',
'public');

        // Création de la voiture
        $vehicule = new Vehicule();
        $vehicule->id = $request->id;
        $vehicule->tarif_journalier = $request->tarifj;
        $vehicule->marque_id = $marque->id;
        $vehicule->modele_id = $modele->id;
        $vehicule->type = $request->type;
        $vehicule->statut = $request->statut;
        $vehicule->transmission = $request->transmission;
        $vehicule->image = $img_vehicule_path;
        $vehicule->save();

        return back()->with('statutAddCar',
trans('gestion_de_la_flotte.statutAddCar'));
    }

    /**
     * Pouvoir supprimer un vehicule
     *
     * @category Gestionnaire
     * @param [int] $id
     * @return \Illuminate\Http\RedirectResponse
     */
    public function deleteCar($id)
    {
        // S'assure que la demande est faisable
        try {
            $vehicule = Vehicule::findOrFail($id);
        } catch (\Throwable $th)
        {
            return back()->with('statutErreur',
trans('gestion_de_la_flotte.statutErreur'));
        }

        // On recupere le nom de l'utilisateur
        $identifiant = $vehicule->id;

        // On supprime l'image lier dans le storage
        if(Storage::disk('public')->exists($vehicule->image))
        {
            Storage::disk('public')->delete($vehicule->image);
        }

        // On supprime l'utilisateur
        $vehicule->delete();

        return back()->with('statutDeleteCar',
trans('gestion_de_la_flotte.statutDeleteCar', ['car' => $identifiant]));
    }

    /**
```

```
     * Mettre à jour le statut d'un véhicule
     *
     * @param Request $request
     * @return \Illuminate\Http\JsonResponse
     */
    public function updateStatut(Request $request)
    {
        // On verifie la requete
        $request->validate([
            'id_vehicule'       => ['required', 'exists:vehicules,id'],
            'statut'            => ['required', Rule::in(VehiculeStatutEnum::DISPO-
>value,VehiculeStatutEnum::REVISION->value,VehiculeStatutEnum::HORSU-
>value,VehiculeStatutEnum::LOUER->value)],
        ]);
        try {
            $vehiculeLouer = Vehicule::findOrFail($request->id_vehicule);
        } catch (\Throwable $th) {
            return response()->json(['erreur' => 'Une erreur est survenue...']);
        }

        // On met a jour le statut du vehicule
        $vehiculeLouer->statut = $request->statut;
        $vehiculeLouer->save();

        // On recupere apres le rafraichissement de la page
        session()->flash('statutCar', trans('gestion_de_la_flotte.statutUpdate'));

        // On retourne la promesse
        return response()->json(['success' => 'Statut mis a jour!']);
    }
}
```

app/Http/Controllers/GdrController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Location;
use App\Models\User;
use App\Models\Vehicule;
use Illuminate\Http\Request;
use App\Enum\UserCategorieEnum;
use App\Enum\VehiculeStatutEnum;
use Carbon\Carbon;
use Illuminate\Support\Facades\Auth;

class GdrController extends Controller
{
    /**
     * On retourne la page Gestion des reservations
     *
     * @return \Illuminate\Contracts\View\View returned
     */
```

```php
    public function index(Request $request)
    {
        if ($request->hasAny(['type','transmission']))
        {
            if (!empty($request->type))
            {
                if (!empty($request->transmission))
                {
                    $vehicules = Vehicule::where([
                        ['type', $request->type],
                        ['transmission', $request->transmission]
                        ])->whereNotIn('statut',
[VehiculeStatutEnum::REVISION,VehiculeStatutEnum::HORSU,VehiculeStatutEnum::LOUER])
                            ->get();
                }
                else
                {
                    $vehicules = Vehicule::where('type', $request->type)
                                            ->whereNotIn('statut',
[VehiculeStatutEnum::REVISION,VehiculeStatutEnum::HORSU,VehiculeStatutEnum::LOUER])
                                            ->get();
                }
            }
            else
            {
                if (!empty($request->transmission))
                {
                    $vehicules = Vehicule::where('transmission', $request-
>transmission)
                                            ->whereNotIn('statut',
[VehiculeStatutEnum::REVISION,VehiculeStatutEnum::HORSU,VehiculeStatutEnum::LOUER])
                                            ->get();
                }
                else
                {
                    $vehicules = Vehicule::whereNotIn('statut',
[VehiculeStatutEnum::REVISION,VehiculeStatutEnum::HORSU,VehiculeStatutEnum::LOUER])
                                            ->get();
                }
            }
            session()->flashInput($request->input());
        }
        else
        {
            $vehicules = Vehicule::whereNotIn('statut',
[VehiculeStatutEnum::REVISION,VehiculeStatutEnum::HORSU,VehiculeStatutEnum::LOUER])
                                ->get();
        }

        // Recuperer les clients
        if (Auth::user()->categorie->value == UserCategorieEnum::GESTION->value)
        {
            $clients = User::where('categorie', UserCategorieEnum::CLIENT)
                            ->orderBy('prenom', 'ASC')
                            ->get();
        }

        return view('app.gestion_des_reservations')
                ->with('vehicules', $vehicules ?? null)
```

```
            ->with('clients', $clients ?? null);
    }

    /**
     * Réserver un véhicule par un gestionnaire ou un client
     *
     * @param Request $request
     * @return \Illuminate\Http\RedirectResponse
     */
    public function bookCar(Request $request)
    {
        // On valide les donnees selon la categorie de l'utilisateur
        if (Auth::user()->categorie->value == UserCategorieEnum::GESTION->value)
        {
            $request->validate([
                'duree_location'    => ['required', 'numeric', 'min:1'],
                'client_id'         => ['required', 'exists:users,id'],
                'vehicule_id'       => ['required', 'exists:vehicules,id'],
            ]);
        }
        else {
            $request->validate([
                'duree_location'    => ['required', 'numeric', 'min:1'],
                'vehicule_id'       => ['required', 'exists:vehicules,id'],
            ]);
        }

        // Calculer la periode de reservation
        $duree_location = $request->duree_location;
        $currentTime = Carbon::now();
        $calculTime = now()->parse($currentTime)->addDays($duree_location);
        $dateDebut = date(date('Y-m-d',strtotime($currentTime)));
        $dateFin = date('Y-m-d',strtotime($calculTime));

        // Creation de la reservation
        $location = new Location();
        $location->vehicule_id = $request->vehicule_id;
        $location->client_id = $request->client_id ?? Auth::user()->id;
        $location->date_debut = $dateDebut;
        $location->date_fin = $dateFin;

        // Mettre a jour le statut du vehicule en louer
        $vehiculeLouer = Vehicule::find($request->vehicule_id);
        $vehiculeLouer->statut = VehiculeStatutEnum::LOUER;

        // Calcul du montant du client
        # Variables importante
        $prixSansReduction = $duree_location*$vehiculeLouer->tarif_journalier;
        $pourcentageSemaine = 10/100;
        $pourcentageMois    = 20/100;

        // On regarde si une reduction s'applique avant le prix final
        if ($duree_location >= 7)
        {
            if ($duree_location >= 30)
            {
                $prixLocation = (($prixSansReduction) - (($prixSansReduction) *
```

```
($pourcentageMois)));
            }
            else {
                $prixLocation = (($prixSansReduction) - (($prixSansReduction) *
($pourcentageSemaine)));
            }
        }
        else {
            $prixLocation = $prixSansReduction;
        }

        // Mettre a jour le montant du client
        $client = User::find($request->client_id ?? Auth::user()->id);
        $client->montant_des_locations = $client->montant_des_locations +
$prixLocation;

        // On sauvegarde le tout dans la base de donnees
        $location->save();
        $vehiculeLouer->save();
        $client->save();

        return back()->with('statutLocation',
trans('gestion_des_reservation.bookCar'));
    }
}
```

app/Http/Controllers/LangController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;

class LangController extends Controller
{
    /**
     * Il retourne la langue souhaite par l'utilisateur
     *
     * @return \Illuminate\Http\Response
     */
    public function change(Request $request)
    {
        app()->setLocale($request->lang);
        session()->put('locale', $request->lang);
        return redirect()->back();
    }
}
```

app/Http/Controllers/TdbController.php

```php
<?php

namespace App\Http\Controllers;

use App\Models\Location;
use App\Models\Vehicule;
use Exception;
use Illuminate\Http\Request;
use Illuminate\Validation\Rule;
use App\Enum\VehiculeStatutEnum;

class TdbController extends Controller
{
    /**
     * On retourne la page Tableau de bord
     *
     * @return \Illuminate\Contracts\View\View returned
     */
    public function index()
    {
        $vehicules = Vehicule::where('statut', VehiculeStatutEnum::LOUER)
                            ->get();
        $locations = Location::all();

        $nbVoituresFlotte = Vehicule::count();
        $nbVoituresLoues = Vehicule::where('statut', VehiculeStatutEnum::LOUER)-
>count();

        if ($nbVoituresLoues == 0)
        {
            $rapport = 0;
        }
        else {
            $rapport = (($nbVoituresLoues / $nbVoituresFlotte) * 100);
        }

        return view('app.tableau_de_bord')
                ->with('vehicules', $vehicules ?? null)
                ->with('locations', $locations ?? null)
                ->with('nbVoituresFlotte', $nbVoituresFlotte ?? null)
                ->with('nbVoituresLoues', $nbVoituresLoues ?? null)
                ->with('rapport', round($rapport,0) ?? null);
    }

    /**
     * Mettre à jour le statut d'un véhicule qui a terminé la date de location
     *
     * @category Gestionnaire
     * @param Request $request
     * @return \Illuminate\Http\JsonResponse
     */
    public function updateStatut(Request $request)
    {
        // On verifie la requete
        $request->validate([
            'id_vehicule'       => ['required', 'exists:vehicules,id'],
            'statut'            => ['required', Rule::in(VehiculeStatutEnum::DISPO-
>value,VehiculeStatutEnum::REVISION->value,VehiculeStatutEnum::HORSU->value)],
```

```
        ]);

        try {
            $vehiculeLouer = Vehicule::findOrFail($request->id_vehicule);
            $location = Location::where('vehicule_id',$request->id_vehicule)-
>firstOrFail();
            throw_if((session('currentDate') ?? now()) <= $location->date_fin, new
Exception('En dessous de la date de fin'));
        } catch (\Throwable $th) {

            // On recupere apres le rafraichissement de la page
            session()->flash('statutErreur', 'Une erreur est survenue...');
            return response()->json(['erreur' => 'Une erreur est survenue...']);
        }

        // On met a jour le statut du vehicule
        $vehiculeLouer->statut = $request->statut;
        $vehiculeLouer->save();

        // Supprimer la location
        $location->delete();

        // On recupere apres le rafraichissement de la page
        session()->flash('statutCar', 'Statut mis a jour!');

        // On retourne la promesse
        return response()->json(['success' => 'Statut mis a jour!']);
    }

    /**
     * Uniquement pour tester la fonctionnalite de la location
     *
     * @param Request $request
     * @return \Illuminate\Http\JsonResponse
     */
    public function changeCurrentDate(Request $request)
    {
        // On verifie la requete
        $request->validate([
            'currentDate'       => ['required', 'date'],
        ]);
        session(['currentDate' => $request->currentDate]);
        return response()->json(['success' => 'Date à jours']);
    }
}
```